

MIE235 Class Notes

Barsam Rahimi

January 2026

Contents

1	Introduction To Numerical Methods	2
1.1	Error and Approximation	3
1.2	Taylor Series Approximation	4
2	Root-Finding Methods	6
2.1	Incremental Search	6
2.2	Bisection Method	6
2.3	Bisection vs. False Position Method	7
2.4	Open Methods: Fixed Point & Newton-Raphson	8
2.5	Convergence Criteria For Open Methods	9
2.6	N-R Multiple Roots	10
2.7	Open Methods: Secant, IQI and Hybrid Methods	10
3	Systems of Linear Equations	12
3.1	Scaling	13
3.2	LU Decomposition	13
3.3	Matrix Inversion	14
3.4	Gauss-Seidel method	15
3.5	Special Matrices and Computational Complexity	17
4	Curve Fitting	19
4.1	Least-Squares Regression	19
4.2	Nonlinear Curve Fitting	21
4.3	Multiple Linear Regression	22
4.4	Polynomial Interpolation	24
4.4.1	Newton Polynomials	24
4.4.2	Lagrange Polynomials	25
4.4.3	Bilinear Interpolation	25
4.4.4	Linear Spline Fitting	26
4.4.5	Quadratic Spline Fitting	27
4.4.6	Cubic Splines	27

5	Numerical Integration	28
5.1	Integration of Data	28
5.1.1	Trapezoidal Rule	28
5.1.2	Simpson's Rules and Multiple Integrals	29
5.2	Integration of Functions	32
5.2.1	Gauss Quadrature	32
5.2.2	Higher Order Gauss Quadrature	34
5.2.3	Romberg's Integration Rule and Richardson Extrapolation	36
6	Ordinary Differential Equations	38
6.1	High-Accuracy Numerical Differentiation	38
6.2	Euler's Method	39
6.3	Heun's method	40
6.4	Runge-Kutta Methods	40
6.5	Higher Order ODEs and System of ODEs	42
6.6	Application to Chaos Theory	43

1 Introduction To Numerical Methods

Consider the following equation:

$$f(x) = x^2 - x - 6$$

The roots of this are by inspection 3 and -2. But what about a polynomial like:

$$f(x) = x^5 - 2.5x^3 + 4x + 6$$

There are no analytic solutions, so we need numerical methods. One way to find the root is to plot the graph which gives us an approximate solution. Consider the equation of motion for free fall:

$$m \frac{dv}{dt} = mg - cv$$

One assumption we are making is the drag coefficient, c , is constant. SO here, v is the dependent variable, t is the independent variable, m, c are parameters and g is the forcing function. Most real-world models will involve simplifications and assumptions. Good assumptions are important. Solving this ODE for the initial condition $v(0) = 0$ gives:

$$v(t) = \frac{mg}{c}(1 - e^{-(c/m)t})$$

Numerical methods play a fundamental role in engineering!

1.1 Error and Approximation

Since we can never get the exact solution with numerical methods, we will have errors. The goal is to minimize errors. Accuracy is how closely a computed or measured value agrees with the true value. Precision is how close individual data points are to each other.

The true error is the difference between the true value and the approximation:

$$E_t = \text{true value} - \text{approximate value}$$

The true error is not what we are interested but the relative error:

$$\epsilon_t = \frac{E_t}{\text{true value}} \times 100\%$$

In numerical methods, however, we rarely know the true value (if we did, what's the point of numerical approximation?). This is used for problems we do know the solution to so that we can test the accuracy of our solution. Otherwise, we use approximate error, where we guess a value and then iterate to improve the approximation.

$$E_a = \text{current approximation} - \text{previous approximation}$$

As $E_a \rightarrow 0$, we converge to a solution. The relative approximate error is then:

$$\epsilon_a = \frac{E_a}{\text{current approximation}} \times 100\%$$

If you know the true value, use the true error! We use approximate error when solving numerically by iteration. We stop when we reach an error tolerance! Define

$$\epsilon_s = \% \text{ error tolerance}$$

Where s means stopping criterion. We stop iterating when $|\epsilon_a| < \epsilon_s$. An exercise is to approximate $e^{0.5}$ with a stopping criteria of $\epsilon_s = 2\%$ using its Taylor expansion:

$$e^x = \sum_{i=0}^n \frac{x^i}{i!}$$

Truncation errors are for approximations that are used to solve exact mathematical operations and procedures. Round-off errors are computer (and people) based which arise as a result of only being able to represent quantities with a finite number of digits!

1.2 Taylor Series Approximation

Recall for the values of a function, we can write a Taylor series:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \dots$$

We assume $f(x)$ is smooth, meaning the derivatives are finite. Note, for a smooth function, this converges to the exact value of $f(x+h)$. Let us make some notation compaction. Let $h = x_{i+1} - x_i$. Then:

$$f_{i+1} = f_i + hf'_i + \dots = \sum_{n=0}^{\infty} \frac{h^n}{n!} f_i^{(n)}$$

So the Taylor expansion of f_{i+1} allows us to approximate it with resolution of h . The more the terms, the better the approximation. If we take k terms, then the truncation error becomes:

$$\text{truncation error for } k\text{-th order approximation} = \sum_{n=k+1}^{\infty} \frac{h^n}{n!} f_i^{(n)}$$

Consider the previous problem, to approximate $e^{0.5}$ with a 2% error. This defines the number of terms we are going to take, and how many we will truncate. For Taylor series, truncation reduces the number of terms. Typically, the greater the h , the more terms we are going to need, and vice versa. The balance between truncation error and size of h is the heart of numerical methods.

The terms we truncate from the Taylor expansion are called the residual. For example, if $n = 2$:

$$f_{i+1} = f_i + hf'_i + \frac{h^2}{2!}f''_i + R_2$$

So R_n is the truncation error of the n -th order approximation. When h is small, then the first term in R_n will dominate the residual. So

$$f_{i+1} \approx f_i + hf'_i + \frac{h^2}{2!}f''_i + \dots + \frac{h^n}{n!}f_i^{(n)} + O(h^{n+1})$$

Where $O(h^{n+1})$ denotes the highest order of magnitude of the residual.

Now suppose we have f_i and f_{i-1} and we are interested in f_{i+1} . We can approximate for first order:

$$f_{i+1} = f_i + hf'_i + O(h^2)$$

Then, we can solve for f'_i :

$$f'_i \approx \frac{f_{i+1} - f_i}{h} + O(h)$$

This is the 1st order forward difference approximation of f' . We can also do the backwards difference where we instead use f_i and f_{i-1} :

$$f'_i \approx \frac{f_i - f_{i-1}}{h} + O(h)$$

These are first order because the truncation error is $O(h^1)$. These are approximations of the first derivative at point (i, f'_i) . Consider the Taylor expansions for both forward and backwards differences:

$$\begin{aligned} f_{i+1} &= f_i + hf'_i + \frac{h^2}{2!}f''_i + \frac{h^3}{3!}f'''_i + \dots \\ f_{i-1} &= f_i - hf'_i + \frac{h^2}{2!}f''_i - \frac{h^3}{3!}f'''_i + \dots \end{aligned}$$

if we add subtract the bottom from the top and take a 2nd order approximation, we get the central difference formula:

$$f'_i \approx \frac{f_{i+1} - f_{i-1}}{2h} + O(h^2)$$

Since h is typically small, $O(h^2)$ means we achieve a much better error compared to forward and backward differences.

We can also find higher order derivative approximations by playing with the Taylor series:

$$f_{i+1} + f_{i-1} = 2f_i + \frac{2h^2}{2!}f''_i + R_2$$

Then we can solve for f''_i :

$$f''_i \approx \frac{f_{i+1} + f_{i-1} - 2f_i}{h^2} + O(h^2)$$

As you can see, for f'' we need 3 points and f' we need 2 points. so for n -th derivative approximation, we need $n + 1$ many points.

2 Root-Finding Methods

We have 3 methods to approach root-finding; brute force methods, bracketing methods, and open methods. Most programs use hybrid methods, meaning they use multiple methods not just one. In 235, we are interested only in real roots of solutions. We have 2 types, algebraic and transcendental equations.

The general approach is the following; consider a function with multiple roots. We search for the location of each root one at a time which brings us to incremental search method. We refine the root value to desired accuracy: bracketing methods where two points are bracketing the root. This always converges but is very slow. We can also search point by point which is faster but could diverge.

2.1 Incremental Search

March along x and check for a change in sign of $f(x)$. When sign changes, we know we crossed the root. So go back and take a smaller step.

The problem with this approach is that if the step is too large, we might miss the point! So to not miss it, we need to calculate the derivative to see where we change sign. Say we want to find x_r which we know is between x_l and x_u . We initialize with the bracketing values. We start at x_l and move incrementally by Δx , searching for a sign change in $f(x)$.

2.2 Bisection Method

First, find an interval which brackets the root: $[x_l, x_u]$. $f(x_l)$ must have a different sign than $f(x_u)$ which means

$$f(x_l)f(x_u) < 0$$

Now, we estimate the location of the root as halfway between x_l and x_u :

$$x_r = \frac{x_l + x_u}{2}$$

Based on the sign, we pick one side or the other. And then, we reset the bracket and repeat. If $f(x_l)f(x_r) > 0$ then we set $x_l = x_r$. If $f(x_l)f(x_r) < 0$ then we set $x_u = x_r$. Ideally, we want $f(x_r)f(x_l) = 0$ but this is very unlikely to happen. So we instead use $|f(x_l)f(x_r)| < \epsilon$ where ϵ is just a small number. Now what is the problem with this? Suppose we have that $f(x_l)f(x_r) = 10^6$. What is a small number? This leaves some

ambiguity. So instead, we use the relative error which we defined earlier. So the stopping criteria is

$$|f(x_l)f(x_r)| < \epsilon_s \implies \left| \frac{\text{current } x_r - \text{previous } x_r}{\text{current } x_r} \right| \times 100\% < \epsilon_s$$

2.3 Bisection vs. False Position Method

False position method will accelerate convergence but will have some drawbacks. The idea is to assume root is where the secant line of x_L, x_U intersects x . Using similar triangles, we can calculate x_{root}

$$\tan \theta = \frac{f_L - 0}{x_{root} - x_L} = \frac{f_U - 0}{x_U - x_{root}}$$

We can solve for x_{root} :

$$x_{root} = \frac{x_U f_L - x_L f_U}{f_L - f_U} = x_U - \frac{f_U(x_L - x_U)}{f_L - f_U}$$

This is a much more clever method and we start off with a better guess. This method, however, is one sided, meaning once side never moves. We can use the modified false position method to change this, where we artificially move one side closer to the root.

Suppose we have endpoints where we change sign. If we have any odd number of roots within this interval, it is possible for us to miss them! We can also have endpoints on the same side where we have an even number of roots in the interval or no roots at all.

What would we do for a double root case such as $f(x) = (x - 3)(x - 3)(x - 4)$? What about discontinuous functions ?

2.4 Open Methods: Fixed Point & Newton-Raphson

Bracket methods always converge, but they do so very slowly. We have a lower and upper x where we know there exists at least 1 root in between. In open methods, we have 1 point and we try to improve on that point. These are much quicker, however, they can diverge.

The fixed-point iteration requires 1 initial guess. The idea is to re formulate $f(x) = 0$ as $x - g(x) = 0$. Thus, $x = g(x)$ will also satisfy the initial problem. In fixed point, we pick an initial guess, x_0 . Then, find $g(x_0)$. Then, $x_1 = g(x_0)$, $x_2 = g(x_1)$, ... and so on. How do we find $g(x)$?

We can either do algebraic manipulation, or we can just straight up add an x to both sides:

$$e^x + \sin(x) = 0 \rightarrow x = e^x + \sin(x) + x \quad x^3 - 3x + 1 = 0 \rightarrow x = \frac{3x - 1}{x^2}$$

We will then stop, when $\epsilon_a < \epsilon_s$. Fixed point is very robust, but not efficient. It will take time. The other option is Newton-Raphson.

Probably the most common root-finding algorithm is Newton-Raphson. The procedure is the following; we have an initial guess of x_i , we find $f(x_i)$ and $f'(x_i)$. Then, we find the solution to $f'(x_i) = 0$. This is the new estimate x_{i+1} !

Recall if we expand $f(x_{i+1})$ we get

$$f(x_{i+1}) \approx f(x_i) + hf'(x_i) + O(h^2), \quad h \equiv x_{i+1} - x_i$$

If we seek the root, then LHS must be 0. Then, we solve for x_{i+1} :

$$0 = f_i + (x_{i+1} - x_i)f'_i + O(h^2) \implies x_{i+1} = x_i - \frac{f_i}{f'_i} + O(h^2)$$

So we find the Newton-Raphson formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Presence of $O(h^2)$ suggests a super linear convergence rate as we get closer to x_{root} . Newton-Raphson converges very fast when it does so, but what about divergence ? It depends on which initial point we choose.

We can also find these to find intersection of two curves f_1 and f_2 :

$$f_1 - f_2 = f(x) = 0 \implies x = g(x)$$

In fact, graphically we are always finding the intersection of $y = x$ and $y = g(x)$ in the fixed point iterations. If we have that $|g'(x)| < 1$ then the solution will converge. When $|g'(x)| > 1$, then the fixed point iteration approach will diverge.

2.5 Convergence Criteria For Open Methods

Recall, with fixed point, we had a linear convergence rate with a wide radius of convergence. If we had that $|g'(x)| < 1$, then it converged. On the other hand, Newton-Raphson had a quadratic convergence rate - it is much faster - where we had that

$$E_{t,i+1} \sim \frac{-f''(x_r)}{2f'(x_r)} E_{t,i}^2$$

it has a narrower radius of convergence. But there are no specific convergence criterion. Recall that

$$E_{t,i+1} = x_r - x_{i+1}$$

We wrote (using the Taylor expansion) that:

$$f'(x_i)E_{t,i+1} + \frac{f''(\xi)}{2!}E_{t,i}^2 = 0$$

by setting $f(x_i) = 0$. So error at $i + 1$ is proportional to the square of the error at i .

However, there are cases where Newton-Raphson diverges. The first case is when we have a point of inflection. Another case is where we have oscillation in the result, which happens near local extrema. Even though analytically you could take a limit, computationally there is no convergence. Another case is if roots are close to each other, such as a sinusoidal function. There is no general convergence criteria for Newton-Raphson. To avoid divergence, we need to detect and correct the state. For example, substitute back in to see if we are actually getting closer, or set an upper limit on the number of iterations. We can also check for zero and near-zero derivative values to ensure we are not too close to extrema.

2.6 N-R Multiple Roots

With Newton-Raphson, what if we have that $f'(x_i) = 0$ at the root ? Then:

$$f_i = f_i^{(2n)} = 0$$

So essentially what about double roots ? What about inflection points ?

$$f_i = f_i^{(2n+1)} = 0$$

One application of this is in stress analysis. To avoid divergence, we introduce a new function:

$$u(x) = \frac{f(x)}{f'(x)}$$

So if $f = 0 \implies u = 0$. Thus, the of f is also a root of u . Now, if we apply N-R to u :

$$x_{i+1} = x_i - \frac{u(x_i)}{u'(x_i)} = x_i - \frac{f(x_i)f'(x_i)}{[f'(x_i)]^2 - f(x_i)f''(x_i)}$$

We can show that this formula will restore a quadratic convergence rate like Newton-Raphson method. This is the modified Newton-Raphson method.

2.7 Open Methods: Secant, IQI and Hybrid Methods

For N-R method, we need the function's derivative, $f'(x)$. But what if this computation is too complicated or expensive ? Then, we will use the forward, backward and central derivative approximations to estimated $f'(x_i)$:

$$f'_i \sim \frac{f_i - f_{i-1}}{h} \sim \frac{f_{i+1} - f_i}{h} \sim \frac{f_{i+1} + f_{i-1} - 2f_i}{h^2}$$

Substituting into x_i from the backwards derivative expression we find:

$$x_{i+1} \sim x_i - \frac{f_i \cdot (x_i - x_{i-1})}{f_i - f_{i-1}}$$

This is the second method. This method depends on the last two root approximations. So we need 2 initial guesses, x_i and x_{i-1} . We can modify this by using the forward derivative instead!

$$f'_i \sim \frac{f(x_i + \delta) - f(x_i)}{\delta} \implies x_{i+1} = x_i - \frac{f_i \delta}{f(x_i + \delta) - f_i}$$

This will require 1 initial guess, but we also need to be wise with our choice of the δ parameter. This is the modified secant method.

It is also possible to do a quadratic secant curve instead of a linear secant curve! Until now, we have assumed the secant has the form $y = mx + b$. What about quadratics? We can represent the second with a quadratic function fit and we estimate the root by its intersection with x-axis, such that $y = ax^2 + bx + c$. This is the Inverse Quadratic method (IQI). One problem that may rise is that we never know for sure if the parabola will cross the x-axis in the first place! But we know it will always cross the y axis. So instead of expressing y as a function of x , we do the opposite! Let

$$x = ay^2 + by + c$$

This inverse parabola will always cross the x axis. We fit a parabola (using Lagrange interpolation) to three points:

$$\{(x_{i-2}, f_{i-2}), (x_{i-1}, f_{i-1}), (x_i, f_i)\}$$

Then we can write the parabola's (tedious and trivial) equation. Since we know the three points, we find the point at which $f_i = 1$.

So far, all of these methods have been supervised, meaning they have been monitored manually for convergence. But professional algorithms must be unsupervised, meaning they need to be as efficient as possible while having sufficient internal tricks to prevent divergence. An example is the *fzero* function in Matlab which uses Brent's method. Here is the algorithm:

1. Start with a bracketed zone (y_0, y_{-1}) (with user providing $f(x)$)
2. When possible, use IQI, otherwise use secant methods if the points are not distinct.
3. When open methods diverge, use one step of bisection method to reduce the bracket.
4. Repeat step 2 until an unacceptable result is produced.

3 Systems of Linear Equations

Most analysis in engineering result in a system of equations. The brute force method is to solve for a variable in one equation and substitute that into the others. We can also write it in matrix form. Graphically, solving a system of 2 variables corresponds to solving the intersection of two lines, and 3 variables corresponds to solving the intersection of 3 planes. If we have parallel lines/planes there are no solutions and if we have coincident lines/planes there are infinitely many solutions. These are called singular systems. Suppose we have a system of two lines which are very very close to being parallel but aren't. These are called ill-conditioned systems. This type comes up when we deal with experimental results which may have very close coefficients that aren't quite the same. So a small change in the coefficients results in a significant change in the solution.

Consider a system $Ax = b$. Then by Cramer's rule:

$$x_1 = \frac{\det \begin{pmatrix} b_1 & a_{12} & a_{13} \\ b_2 & a_{22} & a_{23} \\ b_3 & a_{32} & a_{33} \end{pmatrix}}{\det A}$$

This is very impractical. Consider the two sets of equations:

$$a_{11}x_1 + a_{12}x_2 = b_1$$

$$a_{21}x_1 + a_{22}x_2 = b_2$$

Then we multiply the first one by a_{21} and second by a_{11} to find the solutions. This is equivalent to Cramer and impractical. We can extend this to the Naïve Gauss Elimination. This involves forward elimination and then back substitution. Forward elimination is when we manipulate equations to get rid of the entries below the diagonal, and back is when we manipulate equations to find unknowns. It is Naïve because it ignores problematic cases. Consider a 3 by 3 system:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ \vdots & \vdots & \vdots \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

When we reduce this to an upper triangular matrix we can easily find x_3 . Then we go back and find x_2 and x_1 . One problem is that the determinant is dependent on the scale of the coefficients.

3.1 Scaling

In each row of A , we will normalize the coefficients. Essentially, the coefficients in each row will have the maximum value of 1. So divide each row by the maximum coefficient in that row.

The advantage is that there are no new round off errors which could result from scaling. However, the determinant would now be "un-scaled" if we want to calculate it.

3.2 LU Decomposition

So far, we have been working with naïve Gaussian elimination. LU decomposition is much more efficient. Gaussian elimination enforces that we operate on the matrix and the right hand side simultaneously. Is there a way to solve a system without considering b ? Consider a system $[A]\{X\} = \{B\}$. Suppose we decompose $[A] = [L][U]$. We will define them in the following way:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix} \quad U = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}$$

So L is a lower triangular and U is an upper triangular matrix. We must determine the intermediate vector $D = UX$ such that $LD = B$. Now the strategy is as follows:

$$AX = B \implies ULX = B, \quad LD = B \implies D = L^{-1}B, \quad UX = D \implies X = U^{-1}L^{-1}B$$

This allows us to find X without having to worry about what B is. The matrix U looks very similar to the Gaussian elimination of A . So perhaps U is the same as the $\text{rref}(A)$, but what is L ?:

$$l_{ij} = \frac{a_{ij}}{a_{jj}} \text{ so } l_{21} = \frac{a_{21}}{a_{11}} \quad l_{31} = \frac{a_{31}}{a_{11}} \quad l_{32} = \frac{a_{32}}{a_{22}}$$

So we already have all the factors.

3.3 Matrix Inversion

Consider a system $Ax = b$. If A is non-singular, then $AA^{-1} = I$ where A^{-1} is the inverse matrix. Thus, we have that $x = A^{-1}b$. How do we determine this matrix then? We can use LU decomposition. Let us multiply both sides by the inverse:

$$LUA^{-1} = I$$

Then by forward composition, $LD = I$, $UA^{-1} = D$.

Consider a vector $F = [a, b, c]$. Then the Euclidean norm of the vector is $\|F\|_2 = \sqrt{a^2 + b^2 + c^2}$. The infinity norm L_∞ is defined as $\|F\|_\infty = \max(a, b, c)$. In n dimensions:

$$\|X\|_2 = \sqrt{\sum_{i=1}^n x_i^2} \quad \|X\|_\infty = \max(x_1, \dots, x_n)$$

The Frobenius norm is the analog of the Euclidean norm for matrices:

$$\|A\|_e = \sqrt{\sum_{ij} a_{i,j}^2}$$

The Frobenius norm is computationally expensive. So we can define a new norm called a row-sum norm.

For a matrix, we denote the Row-sum norm as:

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{i,j}|$$

This is widely used because computationally it is a lot cheaper. Then the condition number of a matrix is defined as:

$$Cond(A) = \|A\| \cdot \|A^{-1}\|$$

This, however, requires that each row is scaled!

Consider the very famous ill-conditioned matrix known as the Hilbert matrix:

$$A = \begin{pmatrix} 1 & 1/2 & \dots & 1/n \\ 1/2 & 1/3 & \dots & 1/(n+1) \\ \vdots & \vdots & \ddots & \vdots \\ 1/n & 1/(n+1) & \dots & 1/(2n-1) \end{pmatrix}$$

For Hilbert matrix, $\|A\|_\infty = \max(1.83, 2.17, 2.35) = 2.35$. But then we also have for the inverse that $\|A^{-1}\|_\infty = \max(37, 192, 180) = 192$ which is a very large number.

So we have that $Cond(A) = \|A\|_\infty \cdot \|A^{-1}\|_\infty$. But what matters here is the order of magnitude! What we really care about is $\log(Cond(A))$. In the last case we have that the rounding error is of the order $10^{\log(Cond(A))-t}$ where t is the digit precision of our matrix elements. A matrix is considered ill-conditioned if its condition number is significantly greater than 1. Intuitively, condition number tells us how good our solution is.

3.4 Gauss-Seidel method

We have learned some direct approaches. But what about iterative solutions ? These are more commonly used in applications such as computational fluid dynamics. The above methods required careful pivoting to avoid divide-by-zero or round-off errors. They are also computationally expensive. For example, a CFD problem with only 100k nodes with 4 variables will need about 640GB of RAM. To bypass this, we use iterative methods which are similar to the root finding methods. The iterative approach is the following: making an initial guess, calculating a new estimate, and repeating until the stopping condition is met. The error will be controlled by the number of iterations. Consider a system of 3 equations, $Ax = b$. Assuming diagonal elements are not zero, we can have

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, \quad x_2 = \frac{b_2 - a_{21}x_1 - a_{23}x_3}{a_{22}}, \quad x_3 = \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}}$$

This looks an awful lot like how in fixed point iteration we had $x = g(x)$ and solved for the zeros of this solution.

So how do we approach this? First start with an initial guess of X . Usually, $x_i = 1$. Then calculate the new guess of x_1 via the first equation. Use this plus the previous guesses to calculate a new guess for equation 2. Continue until x_n , then loop back and start again with the new guesses. Repeat until the stopping condition is satisfied. What are the convergence criterion ? In fixed-point iteration, we had that $x = g(x)$ and for the

iterations to converge, we needed $|g'(x)| < 1$. For a 2 by 2 system of nonlinear multivariable equations $x = u(x, y)$, $y = v(x, y)$, it can be shown that if the following conditions are met, convergence is satisfied:

$$\left| \frac{\partial u}{\partial x} \right| + \left| \frac{\partial u}{\partial y} \right| < 1, \quad \left| \frac{\partial v}{\partial x} \right| + \left| \frac{\partial v}{\partial y} \right| < 1$$

What about the case of linear equations ? By Gauss-Seidel:

$$x_1 = \frac{b_1 - a_{12}x_2}{a_{11}} = u(x_1, x_2) \quad \text{and} \quad x_2 = \frac{b_2 - a_{21}x_1}{a_{22}} = v(x_1, x_2)$$

Then by the above condition:

$$\left| \frac{-a_{12}}{a_{11}} \right| < 1, \quad \left| \frac{-a_{21}}{a_{22}} \right| < 1$$

After some simplification, we get that:

$$\Rightarrow |a_{11}| > |a_{12}| \quad |a_{22}| > |a_{21}|$$

So the diagonal values must be larger than the off-diagonal components! In n by n , the diagonal must be bigger than the sum of the off-diagonal components. For a system of n equations:

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}|$$

Systems that satisfy this are called *diagonally dominant*. They will always converge. Note that non-diagonally dominant systems can also converge!

In iterative methods, ordering of the diagonals does matter to guarantee convergence. So a system may diverge by switching its rows or columns. Gauss-Seidel has a wide radius of convergence which allows it to tolerate poor initial guesses. However, it does not always converge. We can't always reorder equations to obtain diagonal dominance. Gauss-Seidel can converge pretty slowly. But we can use relaxation methods to accelerate its convergence.

In relaxation, we modify each current estimate, x_i , with the weighted sum of the current and previous estimate:

$$x_i^{(k+1)} = \lambda x_i^{temp} + (1 - \lambda)x_i^{(k)}$$

where λ is the relaxation factor. When $\lambda \in (0, 1)$ we have under-relaxation. This slows down the iterations to help a non-convergent system converge. It can damp out oscillations in the residuals. Over-relaxation assumes iterations are moving in the right direction but slowly. So it accelerates slow convergence and $\lambda \in (1, 2)$:

Under-Relaxation: slows down iterations $\rightarrow 0 < \lambda < 1$

Over-Relaxation: accelerates convergence $\rightarrow 1 < \lambda < 2$

3.5 Special Matrices and Computational Complexity

Often in engineering, matrices have special properties. For example, in FEM we have banded matrices. In stress-strain tensors we have symmetric matrices. We can apply simplified methods to solve these matrices. Consider a tri-diagonal matrix:

$$\begin{pmatrix} f_1 & g_1 & 0 & 0 & \cdots & 0 \\ e_2 & f_2 & g_2 & 0 & \cdots & 0 \\ 0 & e_3 & f_3 & g_3 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & & e_n & f_n & g_n \end{pmatrix}$$

These are banded. BW is the full bandwidth and HBW is the half-bandwidth. Then:

$$BW = 2HBW + 1, \quad BW = 3 \text{ for tri-diagonal}$$

In general for these matrices, we have that:

$$a_{ij} = 0 \iff |i - j| < BW$$

Another is the skyline which looks like a tri-diagonal but with the upper triangle looking like a skyline, irregular presence of 0s. To deal with skyline matrices, we only store the non-zero values into one long vector. The skyline is preserved for inverse if the structure is symmetric. It takes less storage than full bandwidth.

A sparse matrix is one with 0s on the diagonal and random zeros around in its off-diagonal components. Most engineering matrices are too big to store fully. Most are, however, very sparse. We store only the non-zero entries, similar to skyline.

Symmetric matrices are ones for which $a_{ij} = a_{ji} \forall i, j$. This is equal to its transpose:

$$A = A^T \iff a_{ij} = a_{ji} \forall i, j$$

Thus, we only need half the storage compared to a normal matrix. For full, we need n^2 entries; n on diagonal and $n^2 - n$ on the off-diagonal. But for symmetric, we only need $\frac{1}{2}(n^2 + n) \sim \frac{1}{2}n^2$ for large n . This simplicity of symmetric matrices allows us to simplify their decomposition. This is known as the Cholesky decomposition and it is an efficient method of matrix decomposition for symmetric matrices. Remember that the LU decomposition is:

$$A = LU$$

A symmetric matrix, however, simplifies this to $A = LL^T$. So we can calculate the elements of L directly from the elements of A . For each row $k \in [1, n]$ and column $i \in [1, k - 1]$:

$$l_{ki} = \frac{a_{ki} - \sum_{j=1}^{i-1} l_{kj}l_{ij}}{l_{ii}}, \quad l_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2}$$

Note that l_{kk} could be the square root of a negative number. So how do we go around this ? The Cholesky method only applies to cases where the matrix is a positive definite matrix! Let x be a non-zero vector. Then, the following condition must be true:

$$x^T A x > 0 \quad \forall x \neq \mathbf{0}$$

The better way to find out whether it is positive definite or not, is using eigenvalues. If all eigenvalues of A are real and positive, then A is positive definite.

$$\det(A - \lambda I) = 0, \quad \lambda \in \mathbb{R}^+ \implies A \text{ is a positive definite matrix}$$

For the tri-diagonal matrix, we have that:

$$e_i x_{i-1} + f_i x_i + g_i x_{i+1} = d_i$$

We seek two variables P and Q such that:

$$x_i = P_i x_{i+1} + Q_i \quad x_{i-1} = P_{i-1} x_i + Q_{i-1}$$

Execution time depends on number of mathematical operations. This is measured in Floating-point Operations (FLOPS). Typically, addition/subtraction and multiplication/division take the same amount of time. Each loop runs about n times. So k -nested loops would be n^k .

Forward elimination which involves 3 nested loops will be $n^3 + 3n^2$. Naïve gaussian forward elimination then has $\frac{2}{3}n^3 + O(n^2)$. When n is large, the cubic term will dominate the quadratic. Thus, similar to how truncation error worked for series expansions, this computation has time complexity $O(n^3)$. For backward substitution, we have 2 nested loops, so $O(n^2)$.

4 Curve Fitting

Regression is finding the best curve that represents the general trend of the data. This is applied to noisy data. Interpolation is when the curve passes through each point of the data. This assumes that the data is very precise.

4.1 Least-Squares Regression

The easiest way to approximate 2D data is to fit a line through it. We want the best line to represent the trend of the points. Essentially, for $y = a_0 + a_1x$, what is the best choice of (a_0, a_1) . The difference between the data points is called the error or residual. So the goal is to minimize these residuals:

$$\min \left(\sum_{i=1}^n e_i \right) = \min \left(\sum_{i=1}^n (y_i - a_0 - a_1 x_i) \right)$$

These are skewed. Furthermore, different error signs may cancel with each other. So perhaps it is best to work with the absolute values of errors.

$$\min \left(\sum_{i=1}^n |e_i| \right) = \min \left(\sum_{i=1}^n |y_i - a_0 - a_1 x_i| \right)$$

if we square both sides, then we can get rid of the absolute value:

$$\Rightarrow \min \sum_{i=1}^n e_i^2 = \min \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2$$

To summarize, we set $\frac{\partial f}{\partial x} = 0$ and $\frac{\partial^2 f}{\partial x^2} = 0$ to find 2 equations for minimization of errors! This provides a unique fit for our data. Let:

$$\begin{aligned}
S_r &= \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2 \\
\Rightarrow \frac{\partial S_r}{\partial a_0} &= -2 \sum_{i=1}^n y_i - a_0 - a_1 x_i = 0 \quad \frac{\partial^2 S_r}{\partial a_1^2} = -2 \sum_{i=1}^n (y_i - a_0 - a_1 x_i) x_i = 0 \\
&\Rightarrow \sum y_i - \sum a_0 - \sum a_1 x_i = 0 \quad \sum y_i x_i - \sum a_0 x_i - \sum a_1 x_i^2 = 0 \\
&\Rightarrow \sum y_i - a_0 n - a_1 \sum x_i = 0 \quad \sum y_i x_i - a_0 \sum x_i - a_1 \sum x_i^2 = 0 \\
&\Rightarrow a_0 n - a_1 \sum x_i = \sum y_i \quad a_0 \sum x_i - a_1 \sum x_i^2 = \sum y_i x_i
\end{aligned}$$

solving this system yields the solution

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad a_0 = \bar{y} - a_1 \bar{x} : \bar{y} = \frac{1}{n} \sum_{i \rightarrow n} y_i, \quad \bar{x} = \frac{1}{n} \sum_{i \rightarrow n} x_i$$

But how do we know if it is a good approximation or not? Recall from statistics, the standard deviation s_y tells us about the spread of data about the mean. Define:

$$S_t = \sum_{i=1}^n (y_i - \bar{y})^2$$

Where S_t is the total sum of squares, in other words, it is the variance, and the standard deviation is

$$s_y = \sqrt{\frac{S_t}{n-1}}$$

We can calculate the sum of squares of the residuals of our regression line! The form is very similar between S_t and S_r . The distance from each point to sample mean is S_t where as S_r is the distance from each point to the fitted line. The fitted line standard deviation is

$$s_{y/x} = \sqrt{\frac{S_r}{n-2}}$$

we have $n-2$ because we have 2 parameters, a_0 and a_1 . So for quadratic, it would be $n-3$ and so on. This is the *standard error of the estimate*. We can use both this and the mean for describing goodness of fit. If

we use the initial bet fit to the data, then there is no fitting, so we use S_t . This is magnitude of error prior to regression. S_r is the magnitude of error after regression is done.

The coefficient of determination, R^2 is how much error we have removed through the regression. We define

$$R^2 = \frac{S_t - S_r}{S_t}$$

Since we expected regression to take away error, $S_t > S_r$ is always true. R^2 is the percent of error removed by regression. So if $R^2 = 1$, then we have a perfect fit! The correlation coefficient $r = \sqrt{(S_t - S_r)/S_t}$:

$$r = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{\sqrt{n \sum x_i^2 - (\sum x_i)^2} \sqrt{n \sum y_i^2 - (\sum y_i)^2}}$$

In fact, $r = \sqrt{R^2}$ which yields the CRAZY EQUATION: $r = R$. Usually, people use r in papers rather than R^2 which shows nicer values.

SEARCH UP ANSCOMBE'S QUARTET TO BLOW YOUR MIND.

4.2 Nonlinear Curve Fitting

For certain non-linear fits, we can use linearization to keep the least-squares equation. For example, an exponential can be linearized by logarithms. But how can we modify least-squares to account for polynomial fits ? Consider a quadratic fit:

$$y = a_0 + a_1 x + a_2 x^2$$

Then we find the fit by the same process, computing the sum of the residual errors and minimizing with respect to 3 variables:

$$\begin{aligned} S_r &= \sum_{i=1}^n (y_i - a_0 - a_1 x_i - a_2 x_i^2)^2 \\ \frac{\partial S_r}{\partial a_0} &= 2 \sum_i (y_i - a_0 - a_1 x_i - a_2 x_i^2) \\ \frac{\partial S_r}{\partial a_1} &= 2 x_i \sum_i (y_i - a_0 - a_1 x_i - a_2 x_i^2) \\ \frac{\partial S_r}{\partial a_2} &= 2 x_i^2 \sum_i (y_i - a_0 - a_1 x_i - a_2 x_i^2) \end{aligned}$$

We then re-arrange into a system of equations:

$$\begin{pmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{pmatrix}$$

For a polynomial fit of degree N , we will have an $N + 1 \times N + 1$ system of equations which we will need to solve.

$$\begin{pmatrix} n & \sum x_i & \sum x_i^2 & \cdots & \sum x_i^N \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \cdots & \sum x_i^{N+1} \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 & \cdots & \sum x_i^{N+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^N & \sum x_i^{N+1} & \sum x_i^{N+2} & \cdots & \sum x_i^{2N} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_N \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \\ \vdots \\ \sum x_i^N y_i \end{pmatrix}$$

The R^2 value is then:

$$S_t = \sum_i (y_i - \bar{y})^2 \quad S_r = \sum_i (y_i - a_0 - a_1 x_i - a_2 x_i^2)^2$$

$$\Rightarrow R^2 = \frac{S_t - S_r}{S_t}, \quad s_y = \sqrt{\frac{S_t}{n-1}} \quad s_{y/x} = \sqrt{\frac{S_r}{n-3}}$$

We can solve this with Gauss-Seidel el! If we're lucky, we can use the Thomas algorithm. For the general case,

$$S_r = \sum_{i=1}^n \left(y_i - \sum_{j=0}^N a_j x_i^j \right)^2 \Rightarrow s_{y/x} = \sqrt{\frac{S_r}{n-N+1}}$$

4.3 Multiple Linear Regression

Recall the first equation we made in this class $v = \frac{mg}{c}(1 - e^{(-c/m)t})$. What if we want to regress with respect to more than 1 variable ? Then we want to fit the data into n -planes. So for 2 variables:

$$y = a_0 + a_1 x_1 + a_2 x_2$$

Then the residual is

$$S_r = \sum_{i=1}^n (y_i - a_0 - a_1 x_{1,i} - a_2 x_{2,i})^2$$

setting $\frac{\partial S_r}{\partial a_0}, \frac{\partial S_r}{\partial a_1}, \frac{\partial S_r}{\partial a_2}$ equal to 0 we get the following:

$$for \ y = \sum_{i=0}^n a_i x_i \implies \left\{ \frac{\partial S_r}{\partial a_i} \right\}_{i=0}^n = 0 \implies s_{y/x} = \sqrt{\frac{S_r}{n - (m+1)}}$$

What if y is a power law ? Then we can linearize by doing the following:

$$y = a_0 \prod_{i=1}^n x_i^{a_i} \implies \log(y) = \log(a_0) + \sum_{i=1}^n a_i \log(x_i)$$

in general for a Hilbert space, the function sequence $\{z_i\}_i^N$ can be regressed by the following method:

$$y_i = a_0 + \sum_{i=1}^N a_i z_i, \quad S_r = \sum_{i=1}^N (e_i)^2 \xrightarrow{\partial_{a_i} S=0} \{y\} = [z]\{a\} + \{e\}$$

By some stupid algebra we get:

$$S_r = \{y\}^T \{y\} - 2\{a\}^T [z]^T \{y\} + \{a\}^T [z]^T [z] \{a\}$$

setting $\partial_i S_r = 0$, we get

$$[z]^T [z] \{a\} = [z]^T \{y\}$$

Sometimes we can't manipulate to linear form. For example $f(x) = a_0(1 - e^{-a_1 x}) + e$. We try a Taylor expansion:

$$\begin{aligned} f_{i,j+1} &\approx f_{i,j} + \frac{\partial f_{i,j}}{\partial a_0} \Delta a_0 + \frac{\partial f_{i,j}}{\partial a_1} \Delta a_1 \\ \implies y_i - f_{i,j} &= \frac{\partial f_{i,j}}{\partial a_0} \Delta a_0 + \frac{\partial f_{i,j}}{\partial a_1} \Delta a_1 + e_i \end{aligned}$$

But we know that

$$\Delta a_0 = a_{0,j+1} - a_{0,j} \quad \Delta a_1 = a_{1,j+1} - a_{1,j}$$

We can write this as a system $\{D\} = \{z_j\} \{\Delta a\} + \{e\}$, where $\{D\} = y_i - f_{i,j}$. So we need to solve the following system iteratively until we converge to a solution:

$$[[z_j]^T [z_j]] \{\Delta a\} = \{[z_j]^T \{D\}\}$$

4.4 Polynomial Interpolation

4.4.1 Newton Polynomials

Newton polynomial is kinda like taylor series where derivatives are approximate. The first order newton polynomial is

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0)$$

This is linear interpolation. The term multiplied to $x - x_0$ is the divided difference which is like a generalized slope, may also be denoted as $f[x_0, x_1]$. Then, in general:

$$f_{n+1}(x) = f[x_0] + f_{dd}[x_0, x_1](x - x_0) + f_{dd}[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots + f_{dd}[x_0, \dots, x_n](x - x_0) \dots (x - x_n)$$

$$\rightarrow f_{n+1}(x) = \sum_{i=0}^n \left(f_{dd}[x_0, \dots, x_i] \prod_{j=0}^i (x - x_j) \right)$$

Where we have that:

$$f_{dd}[x_0, \dots, x_{n+1}] = \frac{f[x_0, \dots, x_{n+1}] - f[x_0, \dots, x_n]}{x_{n+1} - x_n}$$

In general, each divided difference is a derivative approximation, hence the similar form of newton polynomials and taylor series.

There is one unique polynomial for $n + 1$ data points. Given $\{x_0, \dots, x_n\}$ and $\{f(x_0), \dots, f(x_n)\}$ we can solve the system:

$$\begin{pmatrix} 1 & x_0 & \dots & x_0^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ \vdots \\ f(x_n) \end{pmatrix}$$

But newton polynomial is more efficient. The truncation error for the n -th order Newton polynomial is calculated by

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \dots (x - x_n)$$

But we cannot estimate $f^{(n+1)}(\xi)$ since it requires $n + 1$ data points. So we essentially need to save one of the data points for estimation.

4.4.2 Lagrange Polynomials

We can express the polynomial interpolation also through Lagrange polynomials. Newton polynomials are computationally more expensive than Lagrange due to all the divided difference calculations. For a linear Lagrange function:

$$f_1(x) = L_0(x)f(x_0) + L_1(x)f(x_1)$$

We must have that at x_0 , $f_1(x_0) = f(x_0)$ and $f_1(x_1) = f(x_1)$. So at x_i , we must have $L_i(x) = 1$ and at all other points 0. So for linear functions,

$$L_0 = \frac{x - x_1}{x_0 - x_1} \quad L_1 = \frac{x - x_0}{x_1 - x_0}$$

and thus

$$f_1(x) = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1)$$

for an n -th order fit, we have

$$L_j(x) = \prod_{i=0, i \neq j}^n \frac{x - x_i}{x_j - x_i} \quad f_n(x) = \sum_{j=0}^n L_j(x) f(x_j)$$

So since Lagrange is so simple and fast, why not always use Lagrange ? The order of the polynomial with Lagrange interpolation is fixed whereas with Newton, we are free to adjust the order as we go along!

What if we want to find x given $f(x)$? Then it is very dangerous because inverse interpolation can lead to very bad result. Data can be unevenly spaced and it can lead to poor results. Thus, we do root finding on our original interpolation. The more even data are separated from each other, the better the fit, especially when swapping variables.

4.4.3 Bilinear Interpolation

Suppose we are given a set of points $\bigcup_{i=0}^n \{(x_i, y_i, f(x_i, y_i))\}$. How do we find an interpolation through this set of points?

First we interpolate through x and keep y fixed, either using Lagrange or Newton. Such that

$$f_{y1}(x) = \left(\frac{x - x_2}{x_1 - x_2} \right) f(x_1, y_1) + \left(\frac{x - x_1}{x_1 - x_2} \right) f(x_2, y_1)$$

$$f_{y2}(x) = \left(\frac{x - x_2}{x_1 - x_2} \right) f(x_1, y_2) + \left(\frac{x - x_1}{x_1 - x_2} \right) f(x_2, y_2)$$

Now, we interpolate these two functions in the y direction!

$$f_{int}(x, y) = \frac{y - y_2}{y_1 - y_2} f_{y1}(x) + \frac{y - y_1}{y_1 - y_2} f_{y2}(x)$$

If we write out the whole thing, it becomes

$$f_{int}(x, y) = \sum_{i=1}^n \prod_{j=1}^i \frac{x - x_j}{x_i - x_j} \frac{y - y_j}{y_i - y_j} f(x_i, y_i)$$

2D Lagrange functions are called shape function and denoted by $\phi_i(x, y)$ where at node i they are 1 and they are 0 everywhere else. Then:

$$f_{int}(x, y) = \sum_{i=0}^n \phi_i(x, y) f(x_i, y_i)$$

4.4.4 Linear Spline Fitting

When curves pass through every point, error or sudden changes can cause oscillation in our interpolation fit, such as the Heaviside function. What if we just pick some of the points and not all ? A linear spline is the simplest possible spline fit. It implements linear interpolation between adjacent points! Techniques like Lagrange and Newton don't work as nicely for functions like Heaviside. But spline fitting is good. Define a window function,

$$W(x; a, b) = \begin{cases} 1 & a < x \leq b \\ 0 & \text{else} \end{cases}$$

Then we essentially do a linear interpolation from x_0 to x_1 :

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0} (x - x_0)$$

and continue this process for all intervals $[x_{i-1}, x_i] \in [a, b]$, until we've interpolated between every two points. Then, the complete spline fit function is:

$$f(x) = \sum_{i=1}^n W(x; x_{i-1}, x_i) f_i(x)$$

Note that each f_i is different from the other, but the window functions are all the same, just shifted. Spline fitting is very simple to do, but it is not a smooth function, and the first derivatives will be discontinuous at knots. We can deal with these discontinuities we can use higher order splines such as quadratic and cubic splines. This requires more data points, however.

How about high order spline fitting ? Linear splines are simple, but they are not smooth, meaning first derivatives are discontinuous at the nodes. To deal with this we take higher order splines! To do this, we not only want the same value at each knot, but also the same derivative. For quadratic splines, we will have continuous slopes, a cubic spline will also be continuous in curvature! These are still easy to calculate, and make observation of discontinuities in 3rd order or higher polynomials easier.

4.4.5 Quadratic Spline Fitting

For $n + 1$ data points, we need n intervals and $3n$ unknowns! We must have function values be equal at interior knots, which gives $2(n - 1)$ conditions. We must also have the first and last functions pass through the endpoints, which adds 2 conditions. We also must have the first derivatives at interior knots be equal, adding another $n - 1$ conditions. We need 1 more equation! So we also assume that the 2nd derivative is zero at the first endpoint to get our last condition. In general:

$$f_i(x) = a_i x^2 + b_i x + c_i$$

This forms a $3n$ by $3n$ system for n many functions.

4.4.6 Cubic Splines

Now we are aiming for a 3rd order polynomial $f_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$. We need $4n$ equations to find the coefficients. we have the same conditions from quadratic spline, but we need additional conditions!

1. We must have function values be equal at interior knots: $2n - 2$
2. First and last functions must pass through the endpoints: 2
3. 1st derivatives at interior knots are equal: $n - 1$
4. 2nd derivatives at interior knots are equal: $n - 1$
5. 2nd derivatives at endpoints are zero: 2

But this is quite challenging to do. Is there perhaps a faster way to do this ?

We can reformulate this to express the function in terms of second derivatives. How so ? We know that the second derivatives are linear. Meaning

$$f_i''(x) = f_i''(x_{i-1}) \frac{x - x_i}{x_{i-1} - x_i} + f_i''(x_i) \frac{x - x_{i-1}}{x_i - x_{i-1}}$$

So why not first find f_i'' by assuming a linear function ? Notice the coefficients are the Lagrange functions! We can then integrate this equation and find the constants by assuming that $f(x)$ must be equal to $f(x_{i-1})$ and $f(x_i)$ at the endpoints! This is a much faster way to find the constants of integration. This system yields a tri-diagonal matrix which we can solve easily using the Thomas algorithm!

5 Numerical Integration

5.1 Integration of Data

5.1.1 Trapezoidal Rule

The Newton-Cotes methods use interpolating polynomials to integrate over data points. The trapezoid rule is one where we apply interpolating linear polynomials!

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0) \quad \int_a^b f(x) dx \approx \int_a^b f_1(x) dx = (b - a) \left(\frac{f(a) + f(b)}{2} \right)$$

All Newton-Cotes methods have the form $(b - a) \cdot \text{something}$. If we wish to divide $[a, b]$ into n segments, then

$$I = \int_a^{x_1} f dx + \int_{x_1}^{x_2} f dx + \dots + \int_{x_{n-1}}^b f dx$$

$$\Rightarrow I \stackrel{\text{trapezoid}}{=} (b - a) \left(\frac{f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)}{2n} \right)$$

After simplifying we get

$$I \approx h \cdot \left(\frac{1}{2} f(x_0) + \sum_{i=1}^{n-1} f(x_i) + \frac{1}{2} f(x_n) \right), \quad h = \Delta x = \frac{b - a}{n}$$

Recall that for a first order Newton polynomial, the truncation error is

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)(x - x_1) \dots (x - x_n)$$

When we integrate R_1 , a first order polynomial's truncation, we get the error of the trapezoid rule to be:

$$E = -\frac{f''(\xi)}{12} (b - a)^3$$

For multi-segment trapezoid rule, the error for each i -th segment is

$$E_i = -\frac{f''(\xi_i)}{12} \left(\frac{b - a}{n} \right)^3$$

When sum over all of them, the full error becomes:

$$E = -\frac{1}{12} \frac{(b - a)^3}{n^2} f''(\xi)$$

Trapezoid rule is easy to code and apply, multi-segment version works over large intervals, but it works best with flat, slowly changing functions. In large n it can require many function evaluations.

5.1.2 Simpson's Rules and Multiple Integrals

Simpson's rule is essentially the same as trapezoidal rule, except it uses higher order polynomials instead of linear fits. Trapezoid rule had $\int f dx \approx \int f_1 dx$. For Simpson's we have $\int f dx \approx \int f_2 dx$. For the interpolation we use Lagrange interpolation. After simplifying we find:

$$I \approx \frac{1}{3} h [f(x_0) + 4f(x_1) + f(x_2)], \quad h = \frac{x_2 - x_0}{2}$$

this is also called Simpson's 1/3 rule. If we use Newton-Cotes formula, we will get

$$I \approx (b - a) \left[\frac{f(x_0) + 4f(x_1) + f(x_2)}{6} \right]$$

This is essentially taking the average height times the width. If we re-write, we can see the weighting on each height:

$$I \approx (b - a) \left[\frac{1}{6} f(x_0) + \frac{4}{6} f(x_1) + \frac{1}{6} f(x_2) \right]$$

where the coefficients of f are the weights. The error for the trapezoid rule was $E_{TR} \sim f''(\xi)(b-a)^3$. Then the error will scale by 2 even though the degree is only increased by 1 which is better than expected. Then

$$E_{1/3} = -\frac{f^{(4)}(\xi)}{2880}(b-a)^5$$

Simpson's 1/3 rule depends on the average fourth derivative and the large constant division which is a lot more accurate than trapezoid rule. Similar to trapezoid rule, we can also improve this by increasing the number of intervals. For n intervals:

$$I \approx \frac{1}{3}h[f(x_0) + 4 \sum_{i \in \text{odd}}^{n-1} f(x_i) + 6 \sum_{j \in \text{even}}^{n-2} f(x_j) + f(x_n)], \quad h = \frac{b-a}{n} = \frac{x_n - x_0}{n}$$

If we use the Newton-Cotes formula with the above expression of h we get:

$$I \approx (b-a) \left[\frac{f(x_0) + 4 \sum_{i \in \text{odd}}^{n-1} f(x_i) + 6 \sum_{j \in \text{even}}^{n-2} f(x_j) + f(x_n)}{3n} \right]$$

Note that when $n = 2$, both of these reduce to Simpson's 1/3 rule. For n intervals, the error of Simpson's rule becomes

$$E \approx -\frac{\overline{f^{(4)}(\xi)}}{180n^4}(b-a)^5$$

Even though Simpson's rule is much more accurate than trapezoid rule, it still requires equally spaced data and requires an even number of segments.

Recall that in the 1/3 rule, we had an odd number of points and we fitted a 2nd order polynomial (quadratic). In the 3/8 rule, we fit a 3rd order polynomial and have an even number of points. The 3/8 rule yields the following formula for 3 segments with $h = (x_3 - x_0)/3$:

$$I \approx \frac{3}{8}h[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$$

It has a single segment truncation of

$$E_{3/8} = -\frac{f^{(4)}(\xi)}{6480}(b-a)^5$$

This is, however, only slightly better than the 1/3 rule. It is only improved by a factor of 2.25. So what is the advantage ? If we have an even number of points, we use a combination of the 1/3 rule and the 3/8 rule. Specifically, multiple segments of 1/3 rule and then for the last interval the 3/8 rule. Usually, we use 3/8 either for the beginning 3 points or the final 3 points.

For evenly spaced points

If $n = 1$, use single segment trapezoid rule. If $n = 2$ use single segment 1/3 rule. If $n = 3$ use the 3/8 rule. If $n > 3$ and even, use multi-segment 1/3. If $n > 3$ and odd, use multi-segment 1/3 plus single segment 3/8 rule.

If we have an unequal spacing, we either apply trapezoid rule for individual segments, or we identify the equally spaced segments and use the above rule for each.

What if we have multiple integrals ? Most engineering problems depend on several variables and demand multiple integrals! How does one evaluate an integral as the following

$$\int_c^d \int_a^b f(x, y) dx dy$$

Suppose we have an array of points $(x_0, \dots, x_3 | y_0, \dots, y_2)$.

Numerical Double Integration

Consider an array of points. Numerically integrate in one direction, while the values of the second variable stay constant along those arrays. Then, numerically integrate with respect to the second variable.

The choice of direction does not matter for multiple integrals. The order of the interpolating polynomial, however, does matter as it affects the truncation of the integration. But how would we extend this to non-rectangular geometries ?

5.2 Integration of Functions

5.2.1 Gauss Quadrature

This is similar to the trapezoidal rule, However, we need to know what $f(x)$ is.

Suppose we wish to evaluate the following integral:

$$\int_a^b f(x) dx$$

We know that by the trapezoid rule, we fit a straight line to the end points of the interval. But what if we can find two points within $[a, b]$ that gives a better estimate ? Assume the following form: $\int_a^b f(x) dx = w_1 f(x_1) + w_2 f(x_2)$ where w_i are the weights and $f(x_i)$ are the values of the function. Now, all we need is to find out what are x_i and w_i .

Assume that the integral can be evaluated exactly for a constant function. Then:

$$\int_a^b 1 dx = x \Big|_a^b = b - a \implies w_1 + w_2 = b - a$$

Then, we assume that this also holds for linear functions:

$$w_1 x_1 + w_2 x_2 = \frac{b^2 - a^2}{2}$$

We can also continue this assumption to second and third order in order to obtain 4 equations for our variables.

We end up with the following 4 equations:

$$\begin{aligned} w_1 + w_2 &= b - a & w_1 x_1 + w_2 x_2 &= \frac{b^2 - a^2}{2} \\ w_1 x_1^2 + w_2 x_2^2 &= \frac{b^3 - a^3}{3} & w_1 x_1^3 + w_2 x_2^3 &= \frac{b^4 - a^4}{4} \end{aligned}$$

x_i are called Gauss points and w_i are called Gauss weights. This is the two points Gauss-Legendre formula:

$$\int_{-1}^1 f(x) dx \approx f(-1/\sqrt{3}) + f(1/\sqrt{3})$$

This has perfect accuracy for polynomials up to $n = 3$ and requires only 2 function evaluations! So how can we make this so it is defined for $[a, b]$ instead of $[-1, 1]$. Suppose

$$\int_a^b f(x) dx = \int_{-1}^1 g(r) dr$$

Consider $x \rightarrow g(r) : x = c_0 + c_1 r$. Then if we plug this in:

$$c_0 = \frac{b+a}{2} \quad c_1 = \frac{b-a}{2} \implies x = \frac{b+a}{2} + \frac{b-a}{2} r, \quad dx = \frac{b-a}{2} dr$$

Then, we have that

$$\int_{-1}^1 x(r) dr = \int_{-1}^1 f(x(r)) \frac{b-a}{2} dr = \frac{b-a}{2} \left(f(x(-1/\sqrt{3})) + f(x(1/\sqrt{3})) \right)$$

Gauss Quadrature In 1 Dimension

For a function $f(x)$, find a transformation $x(r) = \frac{b+a}{2} + \frac{b-a}{2} r$ which then allows us to use the Gauss-Legendre formula:

$$\int_a^b f(x) dx = \int_{-1}^1 f(x(r)) \frac{b-a}{2} dr = \frac{b-a}{2} \left(f\left(x\left(\frac{-1}{\sqrt{3}}\right)\right) + f\left(x\left(\frac{1}{\sqrt{3}}\right)\right) \right)$$

This result is exact with n points, for a polynomial of order $2n - 1$.

For a 3 segment Gauss Quadrature, we need approximately 5 trapezoid rule segments to reach the same error.

These point locations are the roots of the Legendre polynomials $P_n(x)$ which are the solutions of the Legendre differential equation on $[-1, 1]$:

$$(1 - x^2)y'' - 2xy' + n(n+1)y = 0$$

These are where the r_i values come from. This ODE appears frequently in heat conduction when we use spherical coordinates. This method extends very easily to higher dimensional problems.

How can we extend this to higher orders and curved coordinates ?

5.2.2 Higher Order Gauss Quadrature

Recall for 1 dimension, we performed a change of coordinates $x \rightarrow c_0 + c_1 r = \frac{b+a}{2} + \frac{b-a}{2} r$. Then we transform the integral:

$$\int_a^b f(x) dx \rightarrow \int_{-1}^1 f(x(r)) \frac{b-a}{2} dr = \frac{b-a}{2} \int_{-1}^1 f(r) dr$$

And we then apply the Gauss-Legendre formula or approximated using n Gauss points $\int_{-1}^1 f(r) dr \approx \sum_{i=1}^n w_i f(r_i)$ where n is determined by the polynomial order of $f(r)$ and the r_i points are determined by the roots of the n -th Legendre polynomial.

In 2 dimensions, we extend to (r, s) and so on for higher dimensions. Then

$$\int_{-1}^1 \int_{-1}^1 f(r, s) ds dr \approx \sum_{i=1}^n \sum_{j=1}^n w_i w_j f(r_i, s_j) \approx \sum_{k=1}^{n^2} w_k f(r_k, s_k)$$

The first sum is a lot more practical. But what if our domain is not a unit square or cube ?

To deal with this, we use the Jacobian matrix to perform a change of coordinates!

$$\begin{aligned} dx &= \frac{\partial x}{\partial r} dr + \frac{\partial x}{\partial s} ds \\ dy &= \frac{\partial y}{\partial r} dr + \frac{\partial y}{\partial s} ds \\ \Rightarrow \begin{pmatrix} dx \\ dy \end{pmatrix} &= \begin{pmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial s} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial s} \end{pmatrix} \begin{pmatrix} dr \\ ds \end{pmatrix} = \mathcal{J}(r, s) \begin{pmatrix} dr \\ ds \end{pmatrix} \end{aligned}$$

Then,

$$\iint_R f(x, y) dx dy = \iint_{(-1, -1)}^{(1, 1)} f(x(r, s), y(r, s)) |\mathcal{J}(r, s)| dr ds$$

Since $\mathcal{J}(r, s)$ could be a function of r, s it could possibly increase the order of the polynomial we deal with.

For a quadrilateral element,

$$x = c_1 + c_2 r + c_3 s + c_4 rs$$

$$\implies c_0 = (x_1 + x_2 + x_3 + x_4)/4$$

$$c_1 = (x_1 - x_2 - x_3 + x_4)/4$$

$$c_2 = (x_1 + x_2 - x_3 - x_4)/4$$

$$c_3 = (x_1 - x_2 + x_3 - x_4)/4$$

We can write this in the form of a Lagrange interpolation too:

$$x = \frac{1}{4}(1+r)(1+s)x_1 + \frac{1}{4}(1-r)(1+s)x_2 + \frac{1}{4}(1-r)(1-s)x_3 + \frac{1}{4}(1+r)(1-s)x_4$$

Which is the Lagrange polynomial. We can write this also in terms of shape functions ϕ_i :

$$x = \sum_{i=1}^4 \phi_i(r, s)x_i \quad y = \sum_{i=1}^4 \phi_i(r, s)y_i$$

Gauss Quadrature In Higher Dimensions For Quadrilateral

For a function $f(x, y)$, find the transformation: $x = c_1 + c_2r + c_3s + c_4rs$ and $y = d_1 + d_2r + d_3s + d_4rs$, and compute the Jacobian $\mathcal{J}(r, s)$. Then

$$\iint_{\Omega} f(x, y) dA = \int_{-1}^1 \int_{-1}^1 f(x(r, s), y(r, s)) |\mathcal{J}(r, s)| ds dr$$

To find the parametrization coefficients, use:

$$c_1 = (x_1 + x_2 + x_3 + x_4)/4 \quad d_1 = (y_1 + y_2 + y_3 + y_4)/4$$

$$c_2 = (x_1 - x_2 - x_3 + x_4)/4 \quad d_2 = (y_1 - y_2 - y_3 + y_4)/4$$

$$c_3 = (x_1 + x_2 - x_3 - x_4)/4 \quad d_3 = (y_1 + y_2 - y_3 - y_4)/4$$

$$c_4 = (x_1 - x_2 + x_3 - x_4)/4 \quad d_4 = (y_1 - y_2 + y_3 - y_4)/4$$

For a triangular elements, we need to transform

$$\iint_{\Omega} f(x, y) dA = \int_0^1 \int_0^{1-r} f(r, s) |\mathcal{J}(r, s)| ds dr$$

So we must use the parametrization:

$$x = x_1(x_2 - x_1)r + (x_3 - x_2)s = (1 - r - s)x_1 + rx_2 + sx_3$$

$$y = y_1 + (y_2 - y_1)r + (y_3 - y_1)s = (1 - r - s)y_1 + ry_2 + sy_3$$

The Jacobian matrix of this is then calculated and its determinant will ALWAYS be independent of r, s . So the number of gauss points is only dependent on the function itself. The Gauss points for triangular elements are different than those for square elements.

Gauss Quadrature In Higher Dimensions For Triangular Elements

Using different Gauss points than those for quadrilateral elements, and the following parametrization:

$$x = x_1(x_2 - x_1)r + (x_3 - x_2)s = (1 - r - s)x_1 + rx_2 + sx_3$$

$$y = y_1 + (y_2 - y_1)r + (y_3 - y_1)s = (1 - r - s)y_1 + ry_2 + sy_3$$

Transform the integral

$$\iint_{\Omega} f(x, y) dA = \int_0^{-1} \int_0^{1-r} f(r, s) |J(r, s)| ds dr$$

The Jacobian must be independent of r and s !

5.2.3 Romberg's Integration Rule and Richardson Extrapolation

Recall the goal of going from Trapezoid and Simpson's rule to Gauss quadrature, was to minimize the number of segments. Another method is Romberg integration. It is an iterative method based on the multi-segment Trapezoid rule. This is used to integrate functions that are difficult to integrate. The approach is the following: Take two trapezoid estimates using different number of segments, and use them to generate a third, better estimate. The general approach is Richardson extrapolation.

Richardson was a cool guy...

The multi-segment Trapezoid rule is

$$I = \frac{b-a}{n} \frac{1}{2} (f(x_0) + f(x_n)) + 2 \sum_{i=1}^{n-1} f(x_i) - \frac{(b-a)^3}{12n^2} f'' = \frac{h}{2} (f(x_0) + f(x_n)) + 2 \sum_{i=1}^{n-1} f(x_i) - \frac{b-a}{12} h^2 f''$$

we can write the first term as $I_a(h)$ and the second as $E(h)$ such that

$$I_t = I_a(h) + E(h)$$

For two different approximations, meaning two different h values:

$$I_t = I_a(h_1) + E(h_1) = I_a(h_2) + E(h_2)$$

If we take the ratio of the errors:

$$\frac{E(h_1)}{E(h_2)} = \frac{h_1^2}{h_2^2} \implies E(h_1) = \frac{h_1^2}{h_2^2} E(h_2)$$

Substituting this back into the expression for the total integral:

$$I_a(h_1) + E(h_1) = I_a(h_2) + E(h_2) \implies I_a(h_1) + \frac{h_1^2}{h_2^2} E(h_2) = I_a(h_2) + E(h_2)$$

Using this we can solve for the error:

$$E(h_2) = \frac{I_a(h_2) - I_a(h_1)}{\frac{h_1^2}{h_2^2} - 1} \implies I_t = \frac{(h_1^2/h_2^2)I_a(h_2) - I_a(h_1)}{h_1^2/h_2^2 - 1}$$

this is Romberg's formula, and it is a type of Richardson extrapolation. This is not exact because the error term we used is also an approximation! So there are a lot more terms that we did not consider.

But suppose we double the number of segments, meaning $h_1/h_2 = 2$. then,

$$I_t \approx \frac{4}{3}I_a(h_2) - \frac{1}{3}I_a(h_1)$$

We can show that this approximation has an order of $\sim O(h^4)$ where as the multi-step trapezoid rule had an error of $O(h^2)$. So this is a lot better. It is essentially a weighted sum of more and less accurate approximations.

Romberg's Rule and Richardson Extrapolation

For computation of the integral of a function $f(x)$ over some interval, take two trapezoid rule approximations, $I_a(h_1)$ and $I_a(h_2)$ where h_i are the step sizes and $h_1 \neq h_2$. Then,

$$I_t = \frac{(h_1^2/h_2^2)I_a(h_2) - I_a(h_1)}{h_1^2/h_2^2 - 1}$$

If we introduce halving:

$$h_1/h_2 = 2 \implies I_t = \frac{4}{3}I_a(h_1) - \frac{1}{3}I_a(h_2) + O(h_1^4)$$

6 Ordinary Differential Equations

6.1 High-Accuracy Numerical Differentiation

Recall the forward, backward, and central difference formulae:

$$f'_i \approx \frac{f_{i+1} - f_i}{h} \quad f'_i \approx \frac{f_i - f_{i-1}}{h} \quad f'_i \approx \frac{f_{i+1} - f_{i-1}}{h}$$

We derived these formulas using the Taylor expansions back in 1.2. These formulas have error of $O(h)$. The central difference formula (from the series of f_{i+1} and f_{i-1}) is

$$f''_i = \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2} + O(h^2)$$

What if we want a higher order error for the derivative? Using the forward Taylor expansion of $f_{i+2} - 4f_{i+1}$, we find

$$f'_i = \frac{-f_{i+2} + 4f_{i+1} - 3f_i}{2h} + O(h^2)$$

Using $(f_{i+2} + f_{i-2}) - 16(f_{i+1} + f_{i-1})$:

$$f''_i = \frac{-f_{i+2} + 16f_{i+1} - 30f_i + 16f_{i-1} - f_{i-2}}{12h^2} + O(h^4)$$

One can also apply the Richardson extrapolation technique to numerical differentiation by letting $f_{i+1} - f_i/h + O(h) = D_a(h) + E(h)$:

$$D_a(h) = \frac{(h_1/h_2)D_a(h_2) - D_a(h_1)}{h_1/h_2 - 1}$$

As long as you know the nominal error order, Richardson extrapolation works for any numerical method. But what if we have unevenly spaced data ? There are 2 approaches. First, is to use the Taylor expansion with its full form:

$$f_{i+1} = f_i + (x_{i+1} - x_i)f'_i + \frac{(x_{i+1} - x_i)^2}{2}f''_i + \dots$$

and use these formulas to find the answer. The second approach is to fit an interpolating polynomial or spline through the points and then take derivatives. Note that numerical differentiation is very sensitive to noise in the data.

6.2 Euler's Method

Recall the free fall equation: $mg - cv = m \frac{dv}{dt}$. We analytically solved this to obtain $v = \frac{mg}{c}(1 - e^{-ct/m})$. But this is because we used a very simple drag force formula. In reality, this is a lot more complicated:

$$\frac{dv}{dt} = g - \frac{c}{m} \left[v + d \left(\frac{v}{v_{max}} \right)^e \right]$$

This is very difficult to solve analytically. Recall that we also need an initial (or boundary) condition. To use Euler's method: We start at $y(x_i) = y_i$. Then for a step size h , calculate the slope ϕ_i . Then $y_{i+1} = y_i + \phi_i h$. For Runge-Kutta methods, the accuracy of y_{i+1} mainly depends on the choice of slope. For numerical differentiation, we wanted to calculate the slope given the next point, but here it's the opposite. In Euler's method:

$$y' = f(x, y) \implies y'_i = \phi_i \quad y_{i+1} = y_i + \phi_i h = y_i + f(x_i, y_i)h$$

This is the simplest approach. There are better ones that are called predictor-corrector methods. The local truncation error of Euler's method is $O(h^2)$ and its global truncation is $O(h)$.

6.3 Heun's method

Simplest type of predictor-corrector methods. Recall from Euler:

$$y'_i = \frac{y_{i+1} - y_i}{h} = f(x_i, y_i)$$

For higher order ODEs, we still need to start with Euler's method which carries propagated error.

An alternative is the single-step Heun's method, which improves the estimation of the slope over the interval by using more than one estimate.

Heun's Method

Take the slope at start $k_1 = f(x_i, y_i)$. Then, predict using Euler $y_{i+1}^0 = y_i + k_1 h$. Calculate the slope at the end $k_2 = f(x_{i+1}, y_{i+1}^0)$. Then correct by taking the average:

$$y_{i+1} = y_i + \frac{k_1 + k_2}{2} h$$

The local truncation error is $O(h^3)$ due to integration, but the global error is $O(h^2)$. We can also iterate on Heun's method by taking y_i^1 and so on to y_i^n for n iterations. We can continue this iteration until

$$\varepsilon_a = \left| \frac{y_i^n - y_i^{n-1}}{y_i^n} \right| \times 100\%$$

The iterative approach does not give perfect results as it still has local truncation at each step. It is used to overcome poor slope estimates for large h values. It is very dependent on step size and the problem itself.

6.4 Runge-Kutta Methods

Higher order Runge-Kutta methods try to include more derivatives from the Taylor expansion to reduce the error. Consider the general update equation

$$y_{i+1} = y_i + \phi_i(x, y, h) \cdot h$$

The function ϕ_i is the increment function:

$$\phi_i = a_1 k_1 + \cdots + a_n k_n$$

The k values are the slopes between our two points, and ϕ is like a weighted average. But how do we know where to measure each slope and the weighting for each ?

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f(x_i + p_1 h, y_i + q_{11} k_1 h) \\ k_3 &= f(x_i + p_2 h, y_i + q_{21} k_1 h + q_{22} k_2 h) \\ &\vdots \\ k_n &= f(x_i + p_n h, y_i + q_{n-1,1} k_1 h + \cdots + q_{n-1,n-1} k_{n-1} h) \end{aligned}$$

Where the constants p and q are to be determined. Suppose we only had 2 slopes. Then,

$$k_1 = f(x_i, y_i) \quad k_2 = f(x_i + p_1 h, y_i + q_{11} k_1 h) \implies y_{i+1} = y_i + (a_1 k_1 + a_2 k_2) h$$

Then using the second order taylor expansion,

$$a_1 + a_2 = 1, \quad a_2/p_1 = 1/2, \quad a_2 q_{11} = 1/2$$

Using $a_1 = 1 - a_2$, $p_1 = q_{11} = 1/(2a_2)$. If we take $a_2 = 1/2$, then this reduces the second order Runge-Kutta method to:

$$y_{i+1} = y_i + \frac{k_1 + k_2}{2} h$$

which is simply Heun's method! If we take $a_2 = 2/3$, then we get Ralston's method:

$$x_{i+3/4} = x_i + 3h/4, \quad y_{i+3/4}^0 = y_i + \frac{3}{4} k_1 h, \quad k_2 = f(x_{i+3/4}, y_{i+3/4}^0)$$

Higher order Runge-Kutta yields even more accurate results, meaning we have to calculate more slopes. The most popular is the RK4 method, meaning we need 4 k values. The classic one is:

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)h$$

The computational effort is calculated by $n \cdot \#$ steps, where n is the number of function evaluations. Errors flatten with larger number of steps, meaning we are better off with higher order RK methods, but for a quick and dirty calculation Euler's method is best.

6.5 Higher Order ODEs and System of ODEs

We wish to convert an n -th order ODE to a first order ODE. In general, for a mass-spring system, we have the following system:

$$m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx = 0$$

We introduce a new variable: $y = \frac{dx}{dt} \implies \frac{dy}{dt} = \frac{d^2x}{dt^2}$. Then $x' = y(t)$.

$$\implies m \frac{dy}{dt} + cy + kx = 0 \implies y' = \frac{-cy(t) - kx(t)}{m}$$

We have 2 unknowns, x and y . What if we had a 3rd order ?

$$p \frac{d^3x}{dt^3} + m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx = 0$$

Then we make $y = x' \implies y' = x''$, $y'' = x'''$. Then, We let $z = y' \implies z' = y''$. Thus,

$$z'(t) = \frac{-mz(t) - cy(t) - kx(t)}{p}$$

For an n -th order ODE, we need n initial conditions! For the above, we need the conditions $x(t_0)$, $x'(t_0) = y(t_0)$, $x''(t_0) = z(t_0)$ to fully solve.

Solving 2nd and 3rd Order ODEs

For a second order ODE $ax'' + bx' + cx = d$, let $y = x' \implies y' = x''$. Then,

$$y' = \frac{-by - cx - d}{a}, \text{ given } y(t_0), x(t_0) \text{ solve using Euler/RK2/RK4}$$

For a third order ODE $ax''' + bx'' + cx' + dx = f$, let $y = x' \implies y' = x''$, $y'' = x'''$ and $z = y' \implies z' = y''$. Then,

$$z' = \frac{-bz - cy - dx - f}{a}, \text{ given } z(t_0), y(t_0), x(t_0) \text{ solve using Euler/RK2/RK4}$$

From Euler's method we had

$$y' = f(x, y) \implies y_{i+1} = y_i + f(x_i, y_i) \cdot h$$

For solving a single ODE we would use

$$x' = f(t, x) \implies x_{i+1} = x_i + f(t_i, x_i)\Delta t$$

Solving System of ODEs

To a coupled system of ODEs, we would use:

$$x' = f(t, x, y) \implies x_{i+1} = x_i + f(t_i, x_i, y_i)\Delta t$$

$$y' = g(t, x, y) \implies y_{i+1} = y_i + g(t_i, x_i, y_i)\Delta t$$

6.6 Application to Chaos Theory

Recall that Richardson was highly interested in numerical prediction of weather conditions. Eventually Edward Lorenz published the first paper on Chaos and nonlinear dynamics. Imagine a slice of air heated from below and cooled from above with edges at constant temperatures, this is the atmosphere. This is a *natural atmospheric convection effect*. The system of ODEs representing atmospheric convection is the Lorenz

system:

$$x' = \sigma(y - x)$$

$$y' = x(\rho - x) - y$$

$$z' = xy - \beta z$$

Here σ is the Prandtl Number and ρ is the Rayleigh number. x, y, z are the convection rate, horizontal temperature variation, and vertical temperature variations respectively. β is a parameter with respect to the shape of the atmospheric layer. These are famous for showing that certain choices of parameters lead to chaos, in this case those parameters being

$$Ra = \frac{g\beta\Delta T L^3}{\alpha\nu} \quad Pr = \frac{\nu}{\alpha}$$

For certain combinations of parameters, Lorenz found that the solution is highly sensitive to small time changes, leading to *chaos*. Chaos is not random, it is deterministic, but it is extremely sensitive to the initial conditions, mainly due to their nonlinearity. Hence, outcomes are unpredictable, without being random. Short-term predictions are still useful especially with ensemble models. In fact, it is highly effective in risk management. An example is in fluid dynamics; for flow over a spherical object, as we increase the Reynolds number, flow goes from laminar to laminar with flow separation, to laminar with vortex shedding, to flow separation and finally turbulent. This is how chaos emerges. It is still governed by RANS (Reynolds Averaged Numerical Simulation) models, but it is highly sensitive to initial conditions, such as freestream velocity, characteristic length and compressibility.